JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.invoke

Class SwitchPoint

java.lang.Object
    java.lang.invoke.SwitchPoint

```
public class SwitchPoint
extends Object
```

A `SwitchPoint` is an object which can publish state transitions to other threads. A switch point is initially in the *valid* state, but may at any time be changed to the *invalid* state. Invalidation cannot be reversed. A switch point can combine a *guarded pair* of method handles into a *guarded delegator*. The guarded delegator is a method handle which delegates to one of the old method handles. The state of the switch point determines which of the two gets the delegation.

A single switch point may be used to control any number of method handles. (Indirectly, therefore, it can control any number of call sites.) This is done by using the single switch point as a factory for combining any number of guarded method handle pairs into guarded delegators.

When a guarded delegator is created from a guarded pair, the pair is wrapped in a new method handle `M`, which is permanently associated with the switch point that created it. Each pair consists of a target `T` and a fallback `F`. While the switch point is valid, invocations to `M` are delegated to `T`. After it is invalidated, invocations are delegated to `F`.

Invalidation is global and immediate, as if the switch point contained a volatile boolean variable consulted on every call to `M`. The invalidation is also permanent, which means the switch point can change state only once. The switch point will always delegate to `F` after being invalidated. At that point `guardWithTest` may ignore `T` and return `F`.

Here is an example of a switch point in action:

```
MethodHandle MH_strcat = MethodHandles.lookup()
    .findVirtual(String.class, "concat", MethodType.methodType(String.class, Strin
SwitchPoint spt = new SwitchPoint();
assert(!spt.hasBeenInvalidated());
// the following steps may be repeated to re-use the same switch point:
MethodHandle worker1 = MH_strcat;
MethodHandle worker2 = MethodHandles.permuteArguments(MH_strcat, MH_strcat.type(),
MethodHandle worker = spt.guardWithTest(worker1, worker2);
assertEquals("method", (String) worker.invokeExact("met", "hod"));
SwitchPoint.invalidateAll(new SwitchPoint[]{ spt });
assert(spt.hasBeenInvalidated());
assertEquals("hodmet", (String) worker.invokeExact("met", "hod"));
```

Discussion: Switch points are useful without subclassing. They may also be subclassed. This may be useful in order to associate application-specific invalidation logic with the switch point. Notice that there is no permanent association between a switch point and the method handles it produces and consumes. The garbage collector may collect method handles produced or consumed by a switch point independently of the lifetime of the switch point itself.

Implementation Note: A switch point behaves as if implemented on top of `MutableCallSite`, approximately as follows:

```
public class SwitchPoint {
    private static final MethodHandle
        K_true  = MethodHandles.constant(boolean.class, true),
        K_false = MethodHandles.constant(boolean.class, false);
    private final MutableCallSite mcs;
    private final MethodHandle mcsInvoker;
    public SwitchPoint() {
        this.mcs = new MutableCallSite(K_true);
        this.mcsInvoker = mcs.dynamicInvoker();
    }
    public MethodHandle guardWithTest(
            MethodHandle target, MethodHandle fallback) {
        // Note:  mcsInvoker is of type ()boolean.
        // Target and fallback may take any arguments, but must have the same type
        return MethodHandles.guardWithTest(this.mcsInvoker, target, fallback);
    }
    public static void invalidateAll(SwitchPoint[] spts) {
        List<MutableCallSite> mcss = new ArrayList<>();
        for (SwitchPoint spt : spts)  mcss.add(spt.mcs);
        for (MutableCallSite mcs : mcss)  mcs.setTarget(K_false);
        MutableCallSite.syncAll(mcss.toArray(new MutableCallSite[0]));
    }
}
```

Constructor Summary

Constructors

Constructor	Description
`SwitchPoint()`	Creates a new switch point.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`MethodHandle`	`guardWithTest(MethodHandle target, MethodHandle fallback)`	Returns a method handle which always delegates either to the target or the fallback.
`boolean`	`hasBeenInvalidated()`	Determines if this switch point has been invalidated yet.
`static void`	`invalidateAll(SwitchPoint[] switchPoints)`	Sets all of the given switch points into the invalid state.

Methods inherited from class java.lang.Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Detail

SwitchPoint

```
public SwitchPoint()
```

Creates a new switch point.

Method Detail

hasBeenInvalidated

```
public boolean hasBeenInvalidated()
```

Determines if this switch point has been invalidated yet.

Discussion: Because of the one-way nature of invalidation, once a switch point begins to return true for `hasBeenInvalidated`, it will always do so in the future. On the other hand, a valid switch point visible to other threads may be invalidated at any moment, due to a request by another thread.

Since invalidation is a global and immediate operation, the execution of this query, on a valid switchpoint, must be internally sequenced with any other threads that could cause invalidation. This query may therefore be expensive. The recommended way to build a boolean-valued method handle which queries the invalidation state of a switch point `s` is to call `s.guardWithTest` on `constant` true and false method handles.

Returns:

`true if this switch point has been invalidated`

guardWithTest

```
public MethodHandle guardWithTest(MethodHandle target,
                                  MethodHandle fallback)
```

Returns a method handle which always delegates either to the target or the fallback. The method handle will delegate to the target exactly as long as the switch point is valid. After that, it will permanently delegate to the fallback.

The target and fallback must be of exactly the same method type, and the resulting combined method handle will also be of this type.

Parameters:

`target - the method handle selected by the switch point as long as it is valid`

`fallback - the method handle selected by the switch point after it is invalidated`

Returns:

a combined method handle which always calls either the target or fallback

Throws:

NullPointerException - if either argument is null

IllegalArgumentException - if the two method types do not match

See Also:

MethodHandles.guardWithTest(java.lang.invoke.MethodHandle, java.lang.invoke.MethodHandle, java.lang.invoke.MethodHandle)

invalidateAll

```
public static void invalidateAll(SwitchPoint[] switchPoints)
```

Sets all of the given switch points into the invalid state. After this call executes, no thread will observe any of the switch points to be in a valid state.

This operation is likely to be expensive and should be used sparingly. If possible, it should be buffered for batch processing on sets of switch points.

If `switchPoints` contains a null element, a `NullPointerException` will be raised. In this case, some non-null elements in the array may be processed before the method returns abnormally. Which elements these are (if any) is implementation-dependent.

Discussion: For performance reasons, `invalidateAll` is not a virtual method on a single switch point, but rather applies to a set of switch points. Some implementations may incur a large fixed overhead cost for processing one or more invalidation operations, but a small incremental cost for each additional invalidation. In any case, this operation is likely to be costly, since other threads may have to be somehow interrupted in order to make them notice the updated switch point state. However, it may be observed that a single call to invalidate several switch points has the same formal effect as many calls, each on just one of the switch points.

Implementation Note: Simple implementations of `SwitchPoint` may use a private `MutableCallSite` to publish the state of a switch point. In such an implementation, the `invalidateAll` method can simply change the call site's target, and issue one call to synchronize all the private call sites.

Parameters:

switchPoints - an array of call sites to be synchronized

Throws:

NullPointerException - if the switchPoints array reference is null or the array contains a null

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.